

Abstract

Reinforcement Learning (RL) has emerged as a powerful framework for sequential decision-making under uncertainty, making it a natural candidate for applications in algorithmic trading and quantitative finance. This project presents the systematic design, implementation, and conceptual evaluation of a Reinforcement Learning-based trading agent trained on historical financial market data. The work is structured as a progressive learning and development pipeline, beginning with foundational numerical computing tools in Python, advancing through theoretical reinforcement learning constructs such as multi-armed bandits and Markov Decision Processes (MDPs), and culminating in the construction of a custom trading environment with a tabular Q-learning agent.

The primary contribution of this project lies not in achieving state-of-the-art trading performance, but in rigorously formulating the trading problem as an MDP, designing a principled reward structure, and critically examining the limitations and risks inherent in RL-driven financial systems. Emphasis is placed on interpretability, reproducibility, and methodological clarity, making the project suitable for academic evaluation, research internships, and technical portfolios. The report situates the developed system within existing reinforcement learning and financial market literature, discusses design trade-offs, and outlines future directions toward more realistic and robust trading agents.

Chapter 1

Introduction

Algorithmic trading has transformed modern financial markets by enabling automated decision-making at speeds and scales far beyond human capability. Traditional algorithmic strategies often rely on fixed rules, statistical arbitrage, or supervised learning models trained to predict short-term price movements. While effective in certain regimes, such approaches struggle to adapt dynamically to non-stationary market conditions and delayed reward structures that naturally arise in trading.

Reinforcement Learning offers a fundamentally different paradigm. Instead of predicting prices directly, an RL agent learns a policy that maps observed market states to trading actions with the objective of maximizing cumulative reward. This sequential decision-making perspective aligns closely with the realities of trading, where actions taken at one time step influence both future states and future opportunities.

This project was designed as an end-to-end learning exercise that incrementally builds toward an RL-based trading agent. The early stages focus on computational literacy using Python and its numerical ecosystem, ensuring the ability to manipulate market data efficiently. Subsequent stages introduce core reinforcement learning concepts through simplified problems such as multi-armed bandits and finite MDPs, before applying these ideas to the considerably more complex domain of financial markets.

The final outcome is a custom trading environment and a tabular Q-learning agent capable of executing Buy, Sell, and Hold decisions on historical price data. Although deliberately constrained in complexity, this system exposes many of the central challenges in RL trading, including reward sparsity, exploration versus exploitation, and

sensitivity to market noise.

Chapter 2

Financial Markets Background

2.1 Market Dynamics

Financial markets are complex adaptive systems composed of heterogeneous agents with varying objectives, time horizons, and information sets. Asset prices emerge from the interaction of supply and demand, incorporating expectations about future cash flows, macroeconomic conditions, and behavioral factors. Unlike controlled environments typically studied in classical reinforcement learning, markets are only partially observable and are influenced by external factors beyond the control or observation of any single agent.

Price time series exhibit several stylized facts that complicate modeling efforts. These include heavy-tailed return distributions, volatility clustering, and regime shifts. From the perspective of an RL agent, these properties imply that the underlying transition dynamics are non-stationary and stochastic, violating many of the simplifying assumptions commonly made in theoretical RL analyses.

Another critical aspect of market dynamics is the feedback loop created by trading actions themselves. While the small-scale agent considered in this project is assumed to be a price taker, real-world trading systems may influence prices, thereby altering the environment in which they operate. This endogenous interaction further complicates the application of reinforcement learning in finance.

2.2 Challenges of Algorithmic Trading

Algorithmic trading presents several challenges that distinguish it from other RL application domains such as games or robotics. First, reward signals are often noisy and delayed. A profitable trade may only be identifiable after several time steps, and short-term losses may be necessary to achieve long-term gains. This complicates credit assignment and slows learning.

Second, overfitting is a pervasive risk. Financial data is limited, non-stationary, and subject to structural breaks. An agent that performs well on historical data may fail catastrophically when deployed in new market conditions. This necessitates careful evaluation on unseen data and conservative interpretations of observed performance.

Finally, risk management is inseparable from trading. Maximizing raw profit without accounting for drawdowns, volatility, or tail risk can lead to strategies that are unacceptable in practice. Incorporating risk-awareness into reward functions and evaluation metrics is therefore a central concern in RL-based trading research.

Chapter 3

Reinforcement Learning Background

3.1 MDP Formulation

At the core of reinforcement learning lies the Markov Decision Process framework. An MDP is formally defined by a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action space, $P(s'|s, a)$ the state transition probabilities, $R(s, a)$ the reward function, and $\gamma \in [0, 1)$ the discount factor.

In the context of trading, states typically encode information about market prices and possibly the agent's current portfolio position. Actions correspond to trading decisions such as buying, selling, or holding an asset. The transition dynamics are governed by market price evolution, which is stochastic and not directly controllable by the agent.

The Markov assumption implies that the next state depends only on the current state and action, not on the full history. While real financial markets may violate this assumption, it remains a practical and widely adopted modeling choice that enables tractable learning algorithms.

3.2 Exploration vs. Exploitation

A fundamental challenge in reinforcement learning is balancing exploration and exploitation. Exploration refers to taking actions to acquire new information about the environment, while exploitation involves choosing actions believed to yield high rewards based on current knowledge. In trading, excessive exploitation can lead to premature convergence to suboptimal strategies, whereas excessive exploration can incur unnecessary financial losses.

This trade-off is particularly pronounced in financial domains due to the cost of exploratory actions. Unlike simulated environments where exploration is free, each ex-

ploratory trade in a real or realistic market setting carries potential monetary loss. As a result, exploration strategies must be carefully designed and interpreted conservatively.

3.3 Policy-Based vs. Value-Based Methods

Reinforcement learning algorithms can be broadly categorized into value-based and policy-based methods. Value-based approaches, such as Q-learning, aim to estimate the expected return of taking a particular action in a given state. Policies are then derived implicitly by selecting actions that maximize these value estimates.

Policy-based methods, on the other hand, directly parameterize and optimize the policy itself. While these methods offer advantages in continuous action spaces and stochastic policies, they often require more data and are more sensitive to hyperparameter choices.

Given the educational and exploratory nature of this project, a value-based approach using tabular Q-learning was selected. This choice prioritizes interpretability and conceptual clarity over raw performance.

Chapter 4

Problem Statement

The central problem addressed in this project is the formulation and solution of a simplified stock trading task using reinforcement learning. Specifically, the goal is to design an RL agent that interacts with historical price data, makes discrete trading decisions, and learns a policy that improves cumulative returns relative to naive baseline strategies.

The problem is constrained by several deliberate simplifications. The action space is limited to Buy, Sell, and Hold decisions, transaction costs are either simplified or omitted, and the state representation is constructed from readily available price information. These constraints are imposed to isolate the learning dynamics of the RL agent and to avoid confounding factors that obscure conceptual understanding.

Despite these simplifications, the problem captures essential elements of real-world trading, including sequential decision-making, delayed rewards, and exposure to market

uncertainty.

Chapter 5

Project Objectives

The objectives of this project are threefold. First, to develop a strong conceptual understanding of reinforcement learning through incremental exposure to increasingly complex problems, starting from multi-armed bandits and progressing to full MDP-based trading environments.

Second, to implement a complete RL trading pipeline, including data ingestion, environment design, agent training, and evaluation against baseline strategies. Emphasis is placed on correctness, modularity, and transparency rather than optimization for performance.

Third, to critically analyze the limitations, risks, and ethical considerations associated with RL-driven trading systems. By explicitly acknowledging failure modes and potential misuse, the project aims to foster a responsible and research-oriented perspective on algorithmic trading.

Chapter 6

Dataset and Market Data

The trading agent developed in this project operates on historical market data, which serves as a proxy for the environment dynamics encountered during learning. In reinforcement learning for finance, historical data is commonly used despite its limitations, as it provides a controlled and reproducible setting for experimentation. The use of such data implicitly assumes that past market behavior contains informative patterns that may generalize, at least partially, to future conditions.

The market data considered consists primarily of time-series price information, including commonly used fields such as opening price, closing price, adjusted closing

price, and trading volume. These variables are standard in quantitative finance and are widely available from public financial data sources. Adjusted prices are particularly important, as they account for corporate actions such as dividends and stock splits, ensuring temporal consistency in the price series.

It is important to emphasize that the dataset is treated as exogenous and fixed. The agent does not influence prices, and issues such as market impact and liquidity constraints are ignored. This assumption is standard in introductory RL trading studies and allows the learning problem to be framed as a stationary MDP, even though real markets are inherently non-stationary.

Chapter 7

Trading Environment Design

A central contribution of this project is the construction of a custom trading environment that formalizes the interaction between the agent and the market. The environment defines the state transitions, reward signals, and termination conditions that collectively determine the learning dynamics. Careful environment design is critical, as subtle choices can significantly alter agent behavior and learning outcomes.

The environment operates in discrete time steps, each corresponding to a fixed interval in the historical price series. At each step, the agent observes the current state, selects an action, receives a reward, and transitions to the next state. This structure mirrors the standard reinforcement learning interaction loop and enables the application of tabular Q-learning without modification.

7.1 State Representation

The state representation encodes the information available to the agent at each decision point. In this project, states are constructed from market price information and the agent’s current position. Typical components include recent price levels or returns and a binary or categorical indicator representing whether the agent currently holds the asset.

This design reflects a trade-off between expressiveness and tractability. Richer state representations may capture more market structure but quickly lead to state-space explosion in tabular methods. By limiting the state to low-dimensional, interpretable features, the project ensures that learning remains feasible while still exposing the agent to meaningful market signals.

The state is assumed to satisfy the Markov property, meaning that it contains sufficient information to predict future rewards and transitions given an action. While this assumption is an approximation in financial markets, it is necessary for applying standard RL algorithms.

7.2 Action Space

The action space is discrete and consists of three possible actions: Buy, Sell, and Hold. A Buy action transitions the agent into a long position if it is not already holding the asset. A Sell action closes an existing position, realizing any accumulated profit or loss. The Hold action leaves the agent's position unchanged.

This simplified action space abstracts away practical considerations such as order sizing, leverage, and short selling. The primary motivation is to reduce complexity and focus on the temporal decision-making aspect of trading rather than execution mechanics. Such abstractions are common in early-stage RL trading research and educational settings.

7.3 Episode Definition

An episode corresponds to a contiguous segment of historical market data. At the beginning of each episode, the environment is reset, the agent's position is cleared, and cumulative reward is set to zero. The episode terminates when the end of the data segment is reached.

Defining episodes in this manner allows the agent to experience multiple independent trajectories through the data, facilitating learning through repeated exposure. However, it also introduces an artificial episodic structure that does not naturally exist in continuous trading. This limitation is acknowledged and discussed in later sections.

Chapter 8

Reward Function Design

8.1 Mathematical Formulation

The reward function is a critical component of the trading environment, as it directly encodes the agent's objective. In this project, rewards are primarily based on changes

in portfolio value resulting from trading actions. Formally, the reward at time step t can be expressed as

$$r_t = V_{t+1} - V_t,$$

where V_t denotes the portfolio value at time t .

This formulation aligns the agent’s objective with profit maximization. However, it also introduces high variance in rewards, as market price movements are noisy and often dominated by random fluctuations. Such variance can slow learning and encourage unstable policies if not carefully managed.

8.2 Risk-Aware Considerations

Pure profit-based rewards ignore risk, which is unacceptable in most practical trading contexts. While this project does not implement advanced risk-adjusted reward metrics, the design acknowledges their importance. Common alternatives include penalizing large drawdowns, incorporating volatility terms, or optimizing risk-adjusted measures such as the Sharpe ratio.

These alternatives were not implemented primarily to maintain conceptual clarity and avoid introducing additional hyperparameters that complicate interpretation. Nevertheless, their absence represents a significant limitation and motivates future extensions.

Chapter 9

Model Architecture and Algorithms

The trading agent employs tabular Q-learning, a value-based reinforcement learning algorithm. Q-learning seeks to learn an action-value function $Q(s, a)$ that estimates the expected cumulative discounted reward of taking action a in state s and following the optimal policy thereafter.

The Q-value updates follow the standard Bellman equation:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right),$$

where α is the learning rate and γ the discount factor.

Tabular Q-learning was chosen due to its simplicity and interpretability. Each state-action pair has an explicit value estimate, enabling direct inspection of learned

behavior. While this approach does not scale to high-dimensional state spaces, it is well-suited for the constrained environment defined in this project.

Chapter 10

Training Procedure

Training proceeds by iterating over multiple episodes of historical data. During training, the agent employs an ϵ -greedy exploration strategy, selecting a random action with probability ϵ and the greedy action otherwise. This mechanism balances exploration of new actions with exploitation of learned value estimates.

Hyperparameters such as the learning rate, discount factor, and exploration rate are selected conservatively based on standard practice in the reinforcement learning literature. Rather than aggressively tuning these parameters, the project prioritizes stability and reproducibility.

Training is conducted exclusively on a subset of the available data, reserving the remainder for evaluation. This separation is critical to assessing generalization and avoiding misleading conclusions drawn from in-sample performance.

Chapter 11

Evaluation Methodology

Evaluation is performed by deploying the trained agent on unseen market data and observing its behavior over a full episode. Performance is compared conceptually against a Buy-and-Hold baseline strategy, which serves as a simple and widely used benchmark in trading studies.

Rather than focusing on numerical performance metrics, evaluation emphasizes qualitative analysis. This includes examining trading frequency, sensitivity to price movements, and consistency of decision-making. Such analysis provides insights into

the agent’s learned policy without overstating its effectiveness.

Chapter 12

Results Interpretation

The observed behavior of the trained agent reflects both the strengths and limitations of the chosen approach. In some market regimes, the agent learns to avoid unfavorable trades and reduce unnecessary position changes. In others, it exhibits erratic behavior driven by noise in the reward signal.

These outcomes underscore a key lesson of RL trading research: apparent success in isolated scenarios does not imply robustness. The results are best interpreted as evidence that the agent has learned a non-trivial policy, rather than as proof of profitability.

Chapter 13

Limitations and Failure Modes

Several limitations constrain the applicability of the developed system. The most significant is the reliance on historical data as a static environment, which ignores non-stationarity and feedback effects. Additionally, the tabular representation restricts scalability and expressiveness.

Failure modes include overfitting to specific price patterns, excessive sensitivity to hyperparameters, and unstable learning dynamics under noisy rewards. Recognizing

these failure modes is essential for responsible interpretation and further development.

Chapter 14

Ethical and Financial Risks

Automated trading systems pose ethical and financial risks, particularly when deployed without adequate oversight. Poorly designed agents may exacerbate market volatility or incur significant losses. While this project is purely educational, it highlights the importance of rigorous testing, transparency, and risk management in real-world applications.

Chapter 15

Future Improvements

Future extensions of this work could incorporate function approximation through deep neural networks, richer state representations including technical indicators, and explicit modeling of transaction costs. Risk-aware reward functions and more realistic

environment dynamics would further enhance realism.

Chapter 16

Conclusion

This project demonstrates the end-to-end development of a reinforcement learning-based trading agent, emphasizing methodological clarity and critical analysis over performance. By framing trading as an MDP and applying tabular Q-learning, the work provides a concrete illustration of both the promise and pitfalls of RL in finance. The insights gained lay a foundation for more advanced research and responsible application of reinforcement learning in trading systems.